

# PRACTICAL BOOK

M.Sc.(IT)-PART II SEMESTER - III

## MACHINE LEARNING

SUBMITTED BY

**Prajapati Mayur Vishnubhai**

Seat No.

Submitted in partial fulfillment of the requirement for  
Qualifying

M.Sc. Part II Semester III Examination

2026-27



University of Mumbai  
Department of Information Technology

Shailendra Education Society's  
Arts, Commerce & Science College  
S.V. Road, Dahisar (E), Mumbai-400068

**Shailendra Education Society's**  
**ARTS, COMMERCE & SCIENCE COLLEGE**

NAAC Re-accredited 'B++' grade with CGPA score 2.98  
Shailendra Nagar, Dahisar (East), Mumbai - 400 068.



## CERTIFICATE

This is to certify that **Mr. Prajapati Mayur Vishnubhai** of M.Sc. (I.T.) Part II Semester III has completed the practical work in the subject of “**Machine Learning**” during the academic year 2026-27 under the guidance of **Asst. Prof. Archana Singh** being the partial requirement for the fulfillment of the curriculum of Master of Science in Information Technology, University of Mumbai.

**Place: Mumbai**

**Date: \_\_/\_\_/2026**

-----  
Internal Examiner

-----  
External Examiner

-----  
M.Sc. (IT) Coordinator

-----  
College Seal

| Sr. No.   | Practical List                                                                                                                                                                                                                                          | Signature |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1.</b> | <b>Data Pre-processing and Exploration</b>                                                                                                                                                                                                              |           |
| a.        | Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers.                                                                                                                                                                       |           |
| b.        | Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and identify potential features and target variables. Note: Explore Univariate and Bivariate graphs (Matplotlib) and Seaborn for visualization. |           |
| c.        | Create or explore datasets to use all pre-processing routines like label encoding, scaling, and binarization.                                                                                                                                           |           |
| <b>2.</b> | <b>Testing Hypothesis</b>                                                                                                                                                                                                                               |           |
| a.        | Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a CSV file and generate the final specific hypothesis. (Create your dataset)         |           |
| <b>3.</b> | <b>Linear Models</b>                                                                                                                                                                                                                                    |           |
| a.        | Simple Linear Regression — Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE.                                                                         |           |
| b.        | Multiple Linear Regression — Extend linear regression to multiple features. Handle feature selection and potential multicollinearity.                                                                                                                   |           |
| c.        | Regularized Linear Models (Ridge, Lasso, ElasticNet) — Implement regression variants like LASSO and Ridge on any generated dataset.                                                                                                                     |           |
| <b>4.</b> | <b>Discriminative Models</b>                                                                                                                                                                                                                            |           |
| a.        | Logistic Regression — Perform binary classification using logistic regression. Calculate accuracy, precision, recall, and understand the ROC curve.                                                                                                     |           |
| b.        | Implement and demonstrate k-Nearest Neighbor algorithm. Read the training data from a CSV file and build the model to classify a test sample. Print both correct and wrong predictions.                                                                 |           |
| c.        | Build a decision tree classifier or regressor. Control hyperparameters like tree depth to avoid overfitting. Visualize the tree.                                                                                                                        |           |
| d.        | Implement a Support Vector Machine for any relevant dataset.                                                                                                                                                                                            |           |
| e.        | Train a random forest ensemble. Experiment with the number of trees and feature sampling. Compare performance to a single decision tree.                                                                                                                |           |
| f.        | Implement a gradient boosting machine (e.g., XGBoost). Tune hyperparameters and explore feature importance.                                                                                                                                             |           |
| <b>5.</b> | <b>Generative Models</b>                                                                                                                                                                                                                                |           |
| a.        | Implement and demonstrate the working of a Naive Bayesian classifier using a sample data set. Build the model to classify a test sample.                                                                                                                |           |
| b.        | Implement Hidden Markov Models using hmmlearn.                                                                                                                                                                                                          |           |
| <b>6.</b> | <b>Probabilistic Models</b>                                                                                                                                                                                                                             |           |
| a.        | Implement Bayesian Linear Regression to explore prior and posterior distribution.                                                                                                                                                                       |           |
| b.        | Implement Gaussian Mixture Models for density estimation and unsupervised clustering.                                                                                                                                                                   |           |
| <b>7.</b> | <b>Model Evaluation and Hyperparameter Tuning</b>                                                                                                                                                                                                       |           |

|           |                                                                                                                                                        |  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| a.        | Implement cross-validation techniques (k-fold, stratified, etc.) for robust model evaluation.                                                          |  |
| b.        | Systematically explore combinations of hyperparameters to optimize model performance. (Use grid and randomized search)                                 |  |
| <b>8.</b> | <b>Bayesian Learning</b>                                                                                                                               |  |
| a.        | Implement Bayesian Learning using inferences.                                                                                                          |  |
| <b>9.</b> | <b>Deep Generative Models</b>                                                                                                                          |  |
| a.        | Set up a generator network to produce samples and a discriminator network to distinguish between real and generated data. (Use a simple small dataset) |  |
| 10.       | Develop an API to deploy your model and perform predictions                                                                                            |  |

## Practical - 1

### 1. Data Pre-processing and Exploration

a. Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers.

Code :-

```
import pandas as pd
```

```
# Create a CSV file
```

```
data = {  
    "Name": ["Amit", "Riya ", " Rahul", "Priya", "Ankit"],  
    "Age": [20, None, 21, 22, 20],  
    "Marks": [85, 90, None, 95, 250]  
}
```

```
df = pd.DataFrame(data)
```

```
df.to_csv("students.csv", index=False)
```

```
# Load CSV file
```

```
df = pd.read_csv("students.csv")
```

```
print("Original Data")
```

```
print(df)
```

```
# Handle missing values
```

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

```
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

```
# Handle inconsistent formatting
```

```
df["Name"] = df["Name"].str.strip()
```

```
# Remove outlier (Marks > 100)
```

```
df = df[df["Marks"] <= 100]
```

```
print("\nCleaned Data")
```

```
print(df)
```

```
# Save cleaned data
```

```
df.to_csv("cleaned_students.csv", index=False)
```

```
print("\nData cleaned and saved successfully!")
```

output :-

## Original Data

|   | Name  | Age  | Marks |
|---|-------|------|-------|
| 0 | Amit  | 20.0 | 85.0  |
| 1 | Riya  | NaN  | 90.0  |
| 2 | Rahul | 21.0 | NaN   |
| 3 | Priya | 22.0 | 95.0  |
| 4 | Ankit | 20.0 | 250.0 |

## Cleaned Data

|   | Name  | Age   | Marks |
|---|-------|-------|-------|
| 0 | Amit  | 20.00 | 85.0  |
| 1 | Riya  | 20.75 | 90.0  |
| 3 | Priya | 22.00 | 95.0  |

Data cleaned and saved successfully!

b. Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and identify potential features and target variables Note: Explore Univariate and Bivariate graphs (Matplotlib) and Seaborn for visualization.

Code :-

```
# =====  
  
# Load Libraries  
  
# =====  
  
import pandas as pd  
  
import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.datasets import load_iris

# =====

# Load Dataset

# =====

iris = load_iris(as_frame=True)

df = iris.frame

# =====

# Display Dataset

# =====

print("First 5 rows of dataset:")

print(df.head())

# =====

# Descriptive Statistics

# =====

print("\nDescriptive Summary Statistics:")

print(df.describe())

# =====

# Identify Features & Target

# =====

X = df.drop('target', axis=1) # Features

y = df['target']             # Target

print("\nFeatures:")

print(X.columns)
```

```
print("\nTarget Variable:")
```

```
print(y.name)
```

```
# =====
```

```
# Univariate Analysis (Matplotlib)
```

```
# =====
```

```
plt.figure()
```

```
plt.hist(df['sepal length (cm)'])
```

```
plt.title("Histogram of Sepal Length")
```

```
plt.xlabel("Sepal Length (cm)")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

```
# =====
```

```
# Bivariate Analysis (Matplotlib)
```

```
# =====
```

```
plt.figure()
```

```
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
```

```
plt.title("Sepal Length vs Petal Length")
```

```
plt.xlabel("Sepal Length (cm)")
```

```
plt.ylabel("Petal Length (cm)")
```

```
plt.show()
```

```
# =====
```

```
# Univariate Analysis (Seaborn)
```

```
# =====
```

```
plt.figure()
```

```
sns.histplot(df['petal length (cm)'])
```

```
plt.title("Seaborn Histogram of Petal Length")
```

```
plt.show()
```

```
# =====

# Bivariate Analysis (Seaborn)

# =====

plt.figure()

sns.scatterplot(

    x='sepal length (cm)',

    y='petal length (cm)',

    hue='target',

    data=df

)

plt.title("Seaborn Scatter Plot with Target")

plt.show()
```

output :-

First 5 rows of dataset:

|   | sepal length (cm) | sepal width (cm) | ... | petal width (cm) | target |
|---|-------------------|------------------|-----|------------------|--------|
| 0 | 5.1               | 3.5              | ... | 0.2              | 0      |
| 1 | 4.9               | 3.0              | ... | 0.2              | 0      |
| 2 | 4.7               | 3.2              | ... | 0.2              | 0      |
| 3 | 4.6               | 3.1              | ... | 0.2              | 0      |
| 4 | 5.0               | 3.6              | ... | 0.2              | 0      |

[5 rows x 5 columns]

Descriptive Summary Statistics:

|       | sepal length (cm) | sepal width (cm) | ... | petal width (cm) | target     |
|-------|-------------------|------------------|-----|------------------|------------|
| count | 150.000000        | 150.000000       | ... | 150.000000       | 150.000000 |
| mean  | 5.843333          | 3.057333         | ... | 1.199333         | 1.000000   |
| std   | 0.828066          | 0.435866         | ... | 0.762238         | 0.819232   |
| min   | 4.300000          | 2.000000         | ... | 0.100000         | 0.000000   |
| 25%   | 5.100000          | 2.800000         | ... | 0.300000         | 0.000000   |
| 50%   | 5.800000          | 3.000000         | ... | 1.300000         | 1.000000   |
| 75%   | 6.400000          | 3.300000         | ... | 1.800000         | 2.000000   |
| max   | 7.900000          | 4.400000         | ... | 2.500000         | 2.000000   |

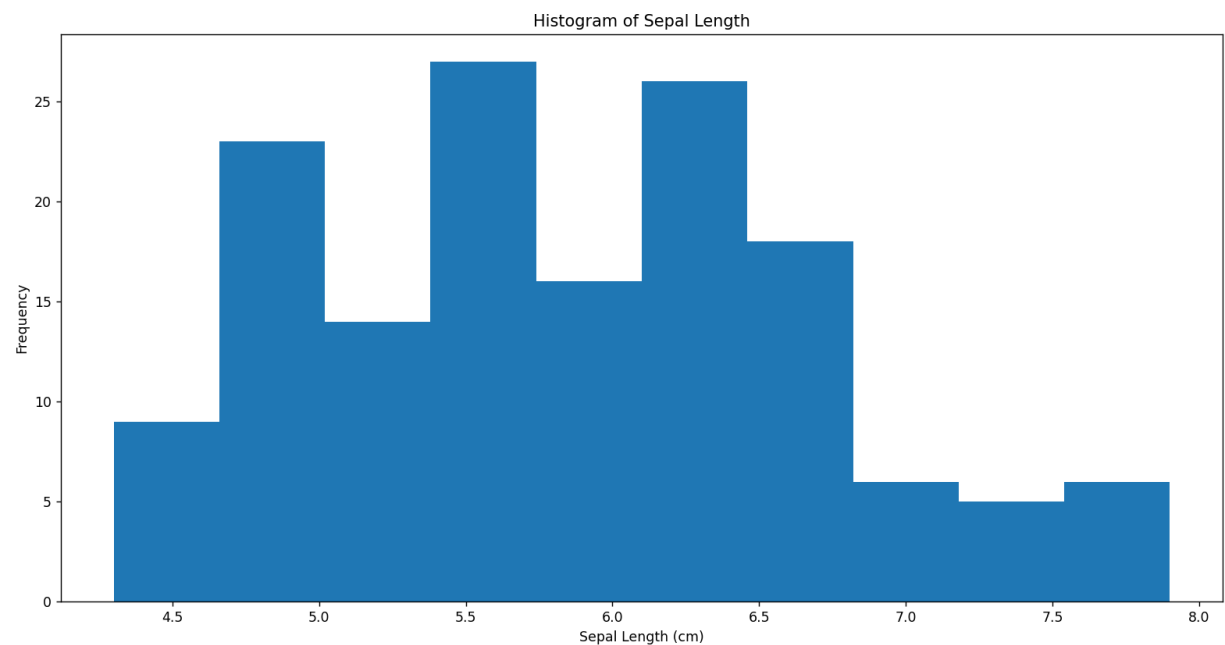
[8 rows x 5 columns]

Features:

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
      'petal width (cm)'],
      dtype='object')
```

Target Variable:

```
target
```



c. Create or Explore datasets to use all pre-processing routines like label encoding, scaling, and binarization.

Code :-

```
# =====  
  
# Import Libraries  
  
# =====  
  
import pandas as pd  
  
from sklearn.preprocessing import LabelEncoder  
  
from sklearn.preprocessing import StandardScaler  
  
from sklearn.preprocessing import Binarizer
```

```

# =====

# Create Dataset

# =====

data = {
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female'],
    'Age': [25, 30, 45, 35, 22],
    'Salary': [50000, 60000, 80000, 65000, 48000],
    'Purchased': [0, 1, 1, 0, 0]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# =====

# Label Encoding

# =====

le = LabelEncoder()
df['Gender_Encoded'] = le.fit_transform(df['Gender'])

print("\nAfter Label Encoding:")
print(df[['Gender', 'Gender_Encoded']])

# =====

# Feature Scaling (Standardization)

# =====

scaler = StandardScaler()
df[['Age_Scaled', 'Salary_Scaled']] = scaler.fit_transform(df[['Age', 'Salary']])

```

```

print("\nAfter Feature Scaling:")

print(df[['Age_Scaled', 'Salary_Scaled']])

# =====

# Binarization

# =====

binarizer = Binarizer(threshold=0.5)

df['Purchased_Binary'] = binarizer.fit_transform(df[['Purchased']])

print("\nAfter Binarization:")

print(df[['Purchased', 'Purchased_Binary']])

```

output :-

Original Dataset:

|   | Gender | Age | Salary | Purchased |
|---|--------|-----|--------|-----------|
| 0 | Male   | 25  | 50000  | 0         |
| 1 | Female | 30  | 60000  | 1         |
| 2 | Female | 45  | 80000  | 1         |
| 3 | Male   | 35  | 65000  | 0         |
| 4 | Female | 22  | 48000  | 0         |

After Label Encoding:

|   | Gender | Gender_Encoded |
|---|--------|----------------|
| 0 | Male   | 1              |
| 1 | Female | 0              |
| 2 | Female | 0              |
| 3 | Male   | 1              |
| 4 | Female | 0              |

After Feature Scaling:

|   | Age_Scaled | Salary_Scaled |
|---|------------|---------------|
| 0 | -0.788742  | -0.917620     |
| 1 | -0.172537  | -0.051941     |
| 2 | 1.676077   | 1.679418      |
| 3 | 0.443667   | 0.380899      |
| 4 | -1.158465  | -1.090756     |

After Binarization:

|   | Purchased | Purchased_Binary |
|---|-----------|------------------|
| 0 | 0         | 0                |
| 1 | 1         | 1                |
| 2 | 1         | 1                |
| 3 | 0         | 0                |
| 4 | 0         | 0                |

## Practical - 2

2. Testing Hypothesis OC1 a. Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a CSV file and generate the final specific hypothesis. (Create your dataset)

Code :-

```
import csv

with open("tennis.csv", "r") as f:

    data = list(csv.reader(f))

h = ['0'] * (len(data[0]) - 1)

for row in data:

    print(row)

    if row[-1] == "True":

        for i in range(len(h)):

            if h[i] == '0':

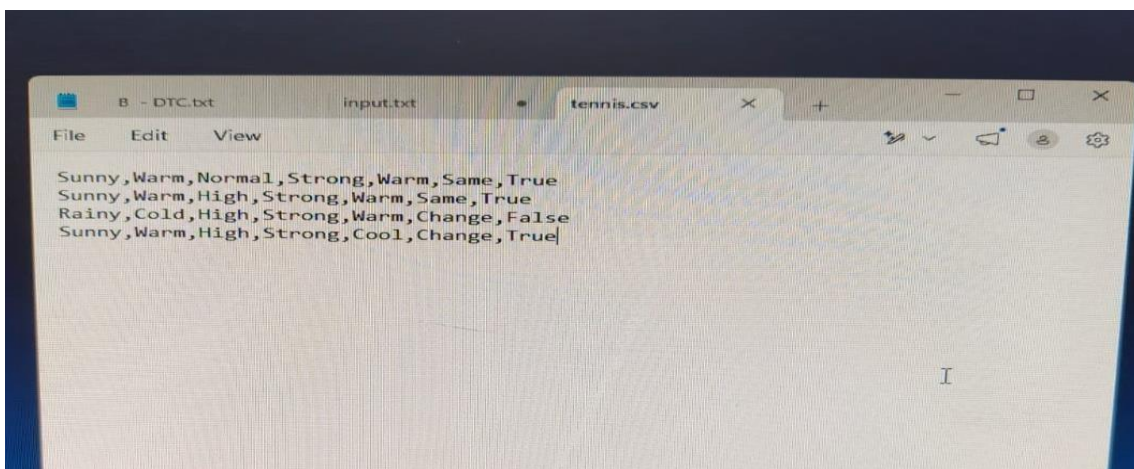
                h[i] = row[i]

            elif h[i] != row[i]:

                h[i] = '?'

print("Most specific hypothesis is")

print([h])
```



output :-

```
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'True']  
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'True']  
['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'False']  
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'True']  
Most specific hypothesis is  
[['Sunny', 'Warm', '?', 'Strong', '?', '?']]
```

## Practical - 3

### 3. Linear Models

a. Simple Linear Regression Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE

Code :-

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

# Dataset

```
X = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
y = np.array([2, 4, 5, 4, 6, 7, 8, 9])
```

# Create and train model

```
model = LinearRegression()
model.fit(X, y)
```

# Prediction

```
y_pred = model.predict(X)
```

# Coefficients

```
print("Intercept:", round(model.intercept_, 2))
print("Coefficient:", round(model.coef_[0], 2))
```

# Regression equation

```
print("\nRegression Equation:")
print(f"y = {model.intercept_:.2f} + {model.coef_[0]:.2f}x")
```

# Performance

```
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
```

```
print("\nMSE:", round(mse, 2))
print("R-squared:", round(r2, 2))
```

# New prediction

```
new_X = np.array([[10]])
prediction = model.predict(new_X)
```

```
print("\nPrediction for X = 10:", round(prediction[0], 2))
```

# -----

# CHART

# -----

```
plt.scatter(X, y, color="blue", label="Actual Data")
```

```
plt.plot(X, y_pred, color="red", linewidth=2, label="Regression Line")
```

```
plt.xlabel("X")
```

```
plt.ylabel("Y")
```

```
plt.title("Simple Linear Regression")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Output :-

Intercept: 1.5

Coefficient: 0.92

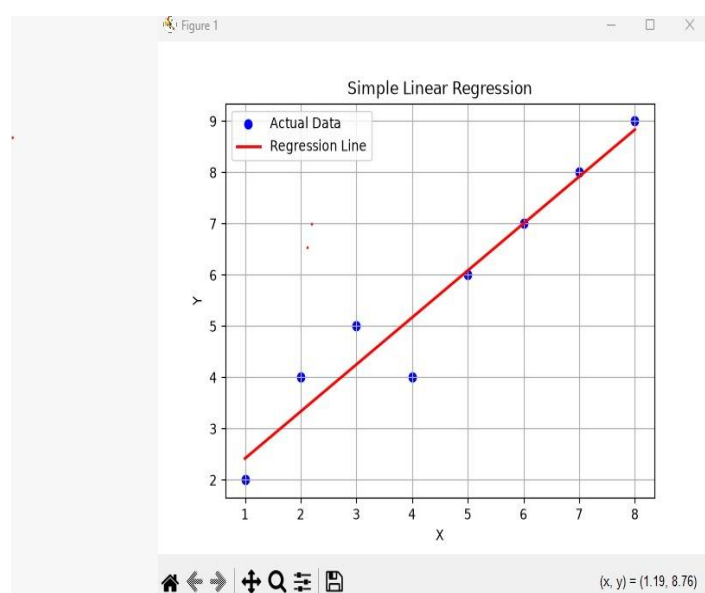
Regression Equation:

$y = 1.50 + 0.92x$

MSE: 0.32

R-squared: 0.93

Prediction for  $X = 10$ : 10.67



b. Multiple Linear Regression Extend linear regression to multiple features. Handle feature selection and potential multicollinearity.

Code :-

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV, Lasso
from sklearn.metrics import r2_score, mean_squared_error
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.feature_selection import SelectFromModel
```

# Step 1: Create dataset

```
np.random.seed(0)
n = 200
X1 = np.random.normal(0, 1, n)
X2 = X1 * 0.95 + np.random.normal(0, 0.1, n)
X3 = X1 * 0.5 + np.random.normal(0, 1, n)
X4 = np.random.normal(0, 1, n)
X5 = np.random.normal(0, 1, n)
y = 3.0 * X1 + 1.5 * X4 + np.random.normal(0, 0.8, n)
```

```
df = pd.DataFrame({'X1': X1, 'X2': X2, 'X3': X3, 'X4': X4, 'X5': X5, 'y': y})

print("=== Dataset Head ===")

print(df.head())


# Step 2: Train-Test Split

X = df[['X1','X2','X3','X4','X5']]

y = df['y']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)


# Step 3: OLS (Statsmodels)

X_train_sm = sm.add_constant(X_train)

ols_model = sm.OLS(y_train, X_train_sm).fit()

print("\n=== OLS Summary ===")

print(ols_model.summary())


# Step 4: Check VIF

vif_data = pd.DataFrame()

vif_data['Feature'] = X_train.columns

vif_data['VIF'] = [variance_inflation_factor(X_train.values, i) for i in range(X_train.shape[1])]

print("\n=== VIF Values ===")

print(vif_data)


# Step 5: Linear Regression

lr = LinearRegression().fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

print("\n=== Linear Regression ===")

print("Coefficients:", lr.coef_)

print("Intercept:", lr.intercept_)

print("R2:", r2_score(y_test, y_pred_lr))
```

```
print("RMSE:", mean_squared_error(y_test, y_pred_lr, squared=False))
```

```
# Step 6: Ridge Regression
```

```
alphas = np.logspace(-3, 3, 50)
```

```
ridge = RidgeCV(alphas=alphas, cv=5).fit(X_train, y_train)
```

```
y_pred_ridge = ridge.predict(X_test)
```

```
print("\n=== Ridge Regression ===")
```

```
print("Alpha:", ridge.alpha_)
```

```
print("Coefficients:", ridge.coef_)
```

```
print("R2:", r2_score(y_test, y_pred_ridge))
```

```
print("RMSE:", mean_squared_error(y_test, y_pred_ridge, squared=False))
```

```
# Step 7: Lasso Regression
```

```
lasso_cv = LassoCV(cv=5, max_iter=5000).fit(X_train, y_train)
```

```
y_pred_lasso = lasso_cv.predict(X_test)
```

```
print("\n=== Lasso Regression ===")
```

```
print("Alpha:", lasso_cv.alpha_)
```

```
print("Coefficients:", lasso_cv.coef_)
```

```
print("R2:", r2_score(y_test, y_pred_lasso))
```

```
print("RMSE:", mean_squared_error(y_test, y_pred_lasso, squared=False))
```

```
# Step 8: Feature Selection
```

```
selector = SelectFromModel(Lasso(alpha=lasso_cv.alpha_))
```

```
selector.fit(X_train, y_train)
```

```
selected = X.columns[selector.get_support()]
```

```
print("\n=== Selected Features ===")
```

```
print(selected.tolist())
```

```
lr_sel = LinearRegression().fit(X_train[selected], y_train)
```

```
y_pred_sel = lr_sel.predict(X_test[selected])
```

```
print("\n=== Final Model ===")
```

```
print("Coefficients:", lr_sel.coef_)
print("Intercept:", lr_sel.intercept_)
print("R2:", r2_score(y_test, y_pred_sel))
print("RMSE:", mean_squared_error(y_test, y_pred_sel, squared=False))

# Step 9: Predictions Compare
print("\n=== Sample Predictions (first 10) ===")
for i in range(10):
    print(f"Actual: {y_test.iloc[i]:.2f} | Pred(LR): {y_pred_lr[i]:.2f} | Ridge: {y_pred_ridge[i]:.2f} | Lasso: {y_pred_lasso[i]:.2f}")
print("\n✅ Done! Multiple Linear Regression executed successfully.")
```

Output :-

```

=== Dataset Head ===
      X1      X2      X3      X4      X5      y
0  1.764052  1.638932  0.283372 -1.550429  1.411172  3.411283
1  0.400157  0.356211 -0.915818  0.417319  0.785804  2.540429
2  0.978738  1.039767  1.256032 -0.944368 -0.057470  1.181809
3  2.240893  2.194375  1.476739  0.238103 -0.391217  7.163606
4  1.867558  1.838193 -0.834759 -1.405963  0.940918  3.676172

=== OLS Summary ===
                                OLS Regression Results
=====
Dep. Variable:                  y      R-squared:                  0.957
Model:                        OLS      Adj. R-squared:            0.956
Method:                    Least Squares      F-statistic:                642.2
Date:                Mon, 15 Dec 2025      Prob (F-statistic):        1.58e-96
Time:                15:28:28      Log-Likelihood:            -168.57
No. Observations:                150      AIC:                        349.1
Df Residuals:                    144      BIC:                        367.2
Df Model:                        5
Covariance Type:                nonrobust
=====
               coef      std err      t      P>|t|      [0.025      0.975]
-----
const          0.0904      0.064      1.407      0.161      -0.037      0.217
X1             3.9428      0.649      6.079      0.000      2.661      5.225
X2            -1.0421      0.678     -1.537      0.127     -2.383      0.298
X3             0.1603      0.063      2.557      0.012      0.036      0.284
X4             1.6494      0.064     25.734      0.000      1.523      1.776
X5             0.0110      0.070      0.156      0.876     -0.128      0.150
=====
Omnibus:                0.344      Durbin-Watson:            1.767
Prob(Omnibus):          0.842      Jarque-Bera (JB):         0.248
Skew:                   0.100      Prob(JB):                 0.883
Kurtosis:               3.001      Cond. No.                  24.1
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

=== VIF Values ===
      Feature      VIF
0      X1  123.821038
1      X2  125.625583
2      X3   1.225009
3      X4   1.026207
4      X5   1.016869

=== Linear Regression ===
Coefficients: [ 3.94277687 -1.04209714  0.16033946  1.64939456  0.01095706]
Intercept: 0.09037620609574197
R2: 0.9461596112545367

```

c. Regularized Linear Models (Ridge, Lasso, ElasticNet) Implement regression variants like LASSO and Ridge on any generated dataset.

Code :-

```

# -----
# REGULARIZED LINEAR MODELS
# Ridge, Lasso, ElasticNet on a generated dataset
# -----

```

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error

# -----
# 1. GENERATE DATASET
# -----
X, y, coef_true = make_regression(
    n_samples=500,
    n_features=10,
    n_informative=5, # 5 actual relevant features
    noise=20,
    coef=True,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# 2. FIT MODELS
# -----
ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.1)
elastic = ElasticNet(alpha=0.1, l1_ratio=0.5)

ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)
elastic.fit(X_train, y_train)

# -----
# 3. PREDICTIONS
# -----
pred_ridge = ridge.predict(X_test)
pred_lasso = lasso.predict(X_test)
pred_elastic = elastic.predict(X_test)

# -----
# 4. PERFORMANCE METRICS
# -----
mse_ridge = mean_squared_error(y_test, pred_ridge)
mse_lasso = mean_squared_error(y_test, pred_lasso)
mse_elastic = mean_squared_error(y_test, pred_elastic)

```

```

print("\n===== MODEL PERFORMANCE (MSE) =====")
print(f"Ridge    MSE: {mse_ridge:.3f}")
print(f"Lasso    MSE: {mse_lasso:.3f}")
print(f"ElasticNet MSE: {mse_elastic:.3f}")

# -----
# 5. COMPARE COEFFICIENTS
# -----

print("\n===== TRUE VS ESTIMATED COEFFICIENTS =====")
print("True Coefs:", coef_true)
print("Ridge Coefs:", ridge.coef_)
print("Lasso Coefs:", lasso.coef_)
print("ElasticNet Coefs:", elastic.coef_)

# -----
# 6. VISUALIZATION (Coefficient shrinkage)
# -----

plt.figure(figsize=(12,5))
plt.plot(coef_true, "ko-", label="True coefficients")
plt.plot(ridge.coef_, "bs-", label="Ridge")
plt.plot(lasso.coef_, "r^-", label="Lasso")
plt.plot(elastic.coef_, "g*-", label="ElasticNet")
plt.title("Coefficient Comparison: Ridge vs Lasso vs ElasticNet")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Magnitude")
plt.grid(True)
plt.legend()
plt.show()

```

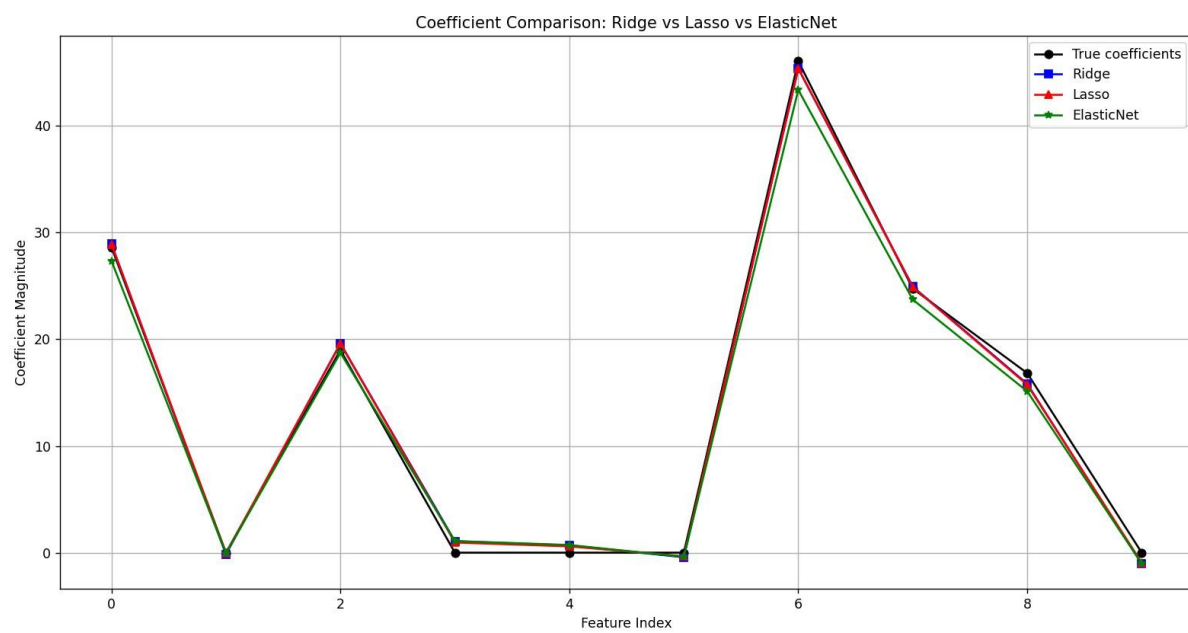
Output :-

```

===== MODEL PERFORMANCE (MSE) =====
Ridge    MSE: 466.987
Lasso    MSE: 466.540
ElasticNet MSE: 469.907

===== TRUE VS ESTIMATED COEFFICIENTS =====
True Coefs: [28.62798621  0.          18.99347437  0.          0.          0.
 46.07121713 24.74629812 16.82365791  0.          ]
Ridge Coefs: [28.9216555  -0.16558146 19.61407315  1.06849384  0.68355722 -0.43094556
 45.36212586 24.93521377 15.8188981  -1.04600487]
Lasso Coefs: [28.89393921 -0.05796303 19.5781544  0.95237845  0.58691157 -0.33532908
 45.39162925 24.89600725 15.73582422 -0.93577407]
ElasticNet Coefs: [27.3419661  -0.06866889 18.72114758  1.09091997  0.71761753 -0.38769599
 43.36880991 23.71659874 15.1192802  -1.01968155]

```



## Practical - 4

### 4 Discriminative Models

a Logistic Regression Perform binary classification using logistic regression. Calculate accuracy, precision, recall, and understand the ROC curve.

Code :-

```
import numpy as np

from sklearn.datasets import make_classification

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_curve, roc_auc_score

import matplotlib.pyplot as plt

# 1. Create a synthetic binary classification dataset

X, y = make_classification(n_samples=1000, n_features=2, n_classes=2,
                          n_informative=2, n_redundant=0, random_state=42)

# 2. Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Train logistic regression model

model = LogisticRegression()

model.fit(X_train, y_train)

# 4. Make predictions

y_pred = model.predict(X_test)

y_proba = model.predict_proba(X_test)[:, 1] # Probability estimates for ROC

# 5. Evaluate performance

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred)

recall = recall_score(y_test, y_pred)
```

```
roc_auc = roc_auc_score(y_test, y_proba)
```

```
print(f"Accuracy: {accuracy:.2f}")
```

```
print(f"Precision: {precision:.2f}")
```

```
print(f"Recall: {recall:.2f}")
```

```
print(f"ROC AUC Score: {roc_auc:.2f}")
```

```
# 6. Plot ROC curve
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
```

```
plt.figure(figsize=(6, 4))
```

```
plt.plot(fpr, tpr, label=f"ROC curve (AUC = {roc_auc:.2f})")
```

```
plt.plot([0, 1], [0, 1], "k--", label="Random Classifier")
```

```
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate (Recall)")
```

```
plt.title("ROC Curve")
```

```
plt.legend(loc="lower right")
```

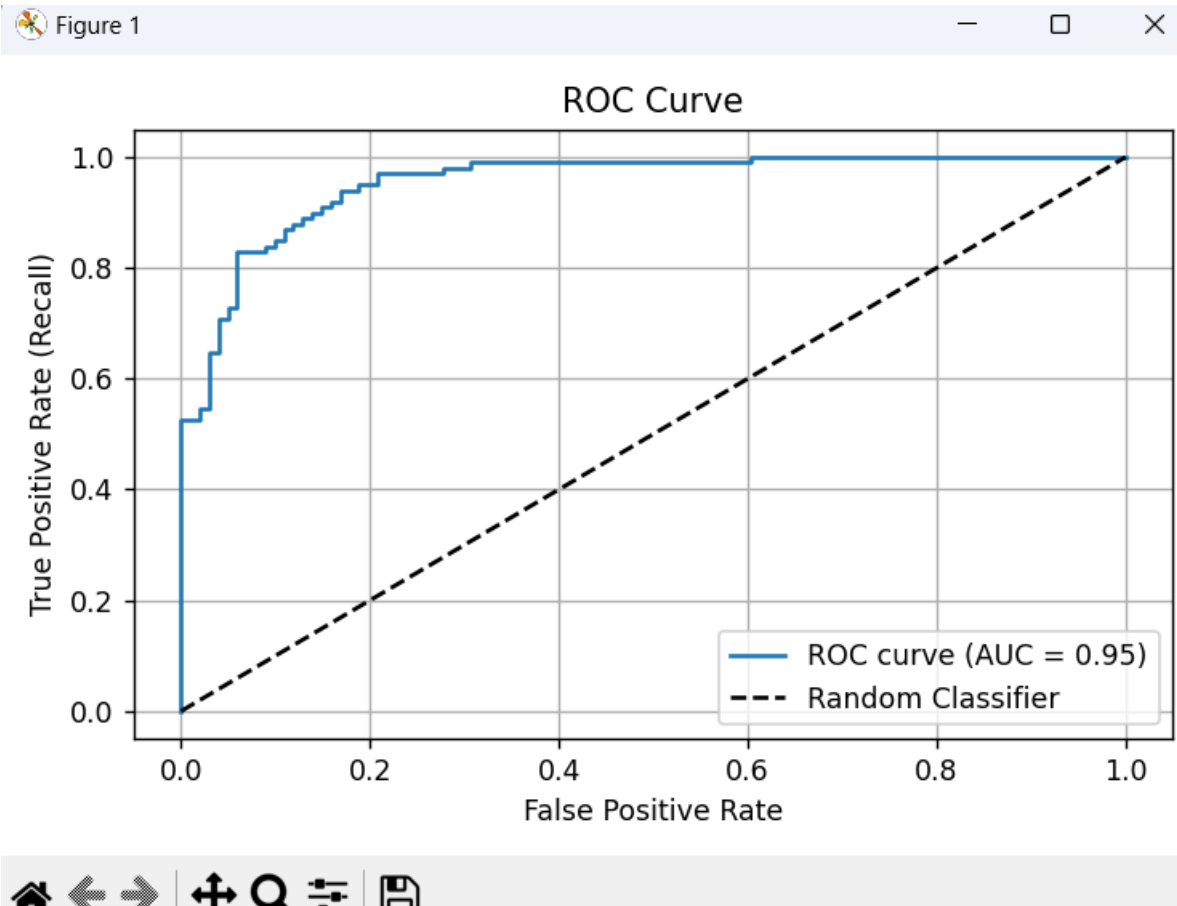
```
plt.grid()
```

```
plt.tight_layout()
```

```
plt.show()
```

output :-

```
Accuracy: 0.88  
Precision: 0.88  
Recall: 0.88  
ROC AUC Score: 0.95
```



b. Implement and demonstrate k-nearest Neighbor algorithm. Read the training data from a .CSV file and build the model to classify a test sample. Print both correct and wrong predictions.

Code :-

B.

```
# =====

# 1. IMPORT LIBRARIES

# =====

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

# =====

# 2. CREATE SAMPLE DATA (instead of CSV)

# =====
```

```
# Example dataset: simple iris-like data
```

```
data = {
```

```
    'sepal_length': [5.1, 4.9, 5.8, 6.0, 6.3, 5.0, 5.5, 6.7, 6.5, 5.7],
```

```
    'sepal_width': [3.5, 3.0, 2.7, 3.4, 3.3, 3.6, 2.8, 3.1, 2.8, 2.5],
```

```
    'petal_length': [1.4, 1.4, 4.1, 4.5, 5.0, 1.5, 4.5, 5.6, 5.2, 4.0],
```

```
    'petal_width': [0.2, 0.2, 1.3, 1.5, 2.0, 0.2, 1.5, 2.4, 2.0, 1.2],
```

```
    'species':    ['setosa','setosa','versicolor','versicolor','virginica',
```

```
                  'setosa','versicolor','virginica','virginica','versicolor']
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# =====
```

```
# 3. SPLIT FEATURES & TARGET
```

```
# =====
```

```
X = df.drop('species', axis=1)
```

```
y = df['species']
```

```
# Split into train & test data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# =====
```

```
# 4. BUILD KNN MODEL
```

```
# =====
```

```
k = 3
```

```
knn = KNeighborsClassifier(n_neighbors=k)
```

```
knn.fit(X_train, y_train)
```

```
# =====
```

```
# 5. PREDICTION
```

```
# =====
```

```
y_pred = knn.predict(X_test)
```

```
# =====
```

```
# 6. PRINT RESULTS
```

```
# =====  
  
print("Test Samples:")  
print(X_test)  
  
print("\nActual Labels:")  
print(y_test.values)  
print("\nPredicted Labels:")  
print(y_pred)  
  
# Identify correct and wrong predictions  
correct = y_test[y_test == y_pred]  
wrong = y_test[y_test != y_pred]  
print("\nCorrect Predictions:")  
print(correct)  
print("\nWrong Predictions:")  
print(wrong)  
  
# Accuracy  
accuracy = accuracy_score(y_test, y_pred)  
print(f"\nModel Accuracy: {accuracy*100:.2f}%")  
  
output :-
```

```
Test Samples:
      sepal_length  sepal_width  petal_length  petal_width
8              6.5           2.8           5.2           2.0
1              4.9           3.0           1.4           0.2
5              5.0           3.6           1.5           0.2
```

```
Actual Labels:
['virginica' 'setosa' 'setosa']
```

```
Predicted Labels:
['virginica' 'versicolor' 'versicolor']
```

```
Correct Predictions:
8    virginica
Name: species, dtype: object
```

```
Wrong Predictions:
1    setosa
5    setosa
Name: species, dtype: object
```

```
Model Accuracy: 33.33%
```

c. Build a decision tree classifier or regressor. Control hyperparameters like tree depth to avoid overfitting. Visualize the tree.

Code :-

```
# =====

# 1. IMPORT LIBRARIES

# =====

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score

import matplotlib.pyplot as plt

# =====

# 2. CREATE SAMPLE DATA

# =====

# Simple dataset (like Iris)

data = {

    'sepal_length': [5.1, 4.9, 5.8, 6.0, 6.3, 5.0, 5.5, 6.7, 6.5, 5.7],
```

```
'sepal_width': [3.5, 3.0, 2.7, 3.4, 3.3, 3.6, 2.8, 3.1, 2.8, 2.5],  
'petal_length': [1.4, 1.4, 4.1, 4.5, 5.0, 1.5, 4.5, 5.6, 5.2, 4.0],  
'petal_width': [0.2, 0.2, 1.3, 1.5, 2.0, 0.2, 1.5, 2.4, 2.0, 1.2],  
'species':    ['setosa','setosa','versicolor','versicolor','virginica',  
               'setosa','versicolor','virginica','virginica','versicolor']  
}
```

```
df = pd.DataFrame(data)
```

```
# =====
```

```
# 3. SPLIT FEATURES & TARGET
```

```
# =====
```

```
X = df.drop('species', axis=1)
```

```
y = df['species']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# =====
```

```
# 4. BUILD DECISION TREE CLASSIFIER
```

```
# =====
```

```
# Control max_depth to avoid overfitting
```

```
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
```

```
tree.fit(X_train, y_train)
```

```
# =====
```

```
# 5. PREDICT & EVALUATE
```

```
# =====
```

```
y_pred = tree.predict(X_test)
```

```
print("Actual Labels:")
```

```
print(y_test.values)
```

```
print("\nPredicted Labels:")
```

```
print(y_pred)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"\nModel Accuracy: {accuracy*100:.2f}%")
```

```
# =====
```

```
# 6. VISUALIZE THE TREE
```

```
# =====
```

```
plt.figure(figsize=(12,8))
```

```
plot_tree(tree, feature_names=X.columns, class_names=tree.classes_, filled=True, rounded=True)
```

```
plt.title("Decision Tree Visualization")
```

```
plt.show()
```

output :-

Actual Labels:

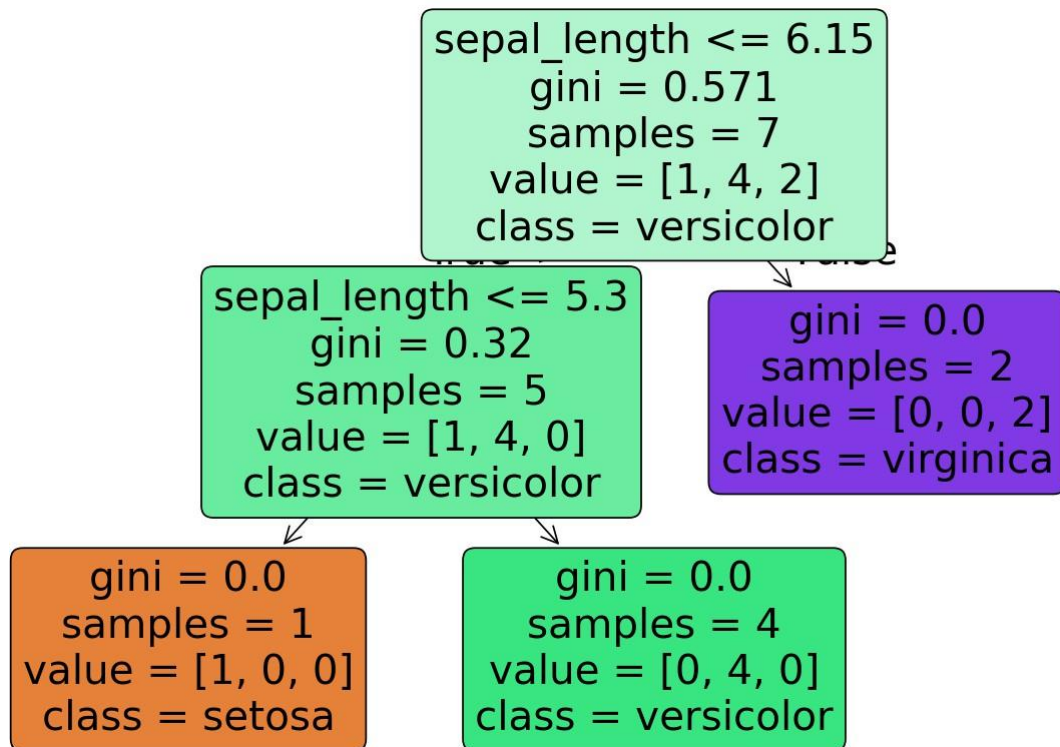
```
['virginica' 'setosa' 'setosa']
```

Predicted Labels:

```
['virginica' 'setosa' 'setosa']
```

Model Accuracy: 100.00%

Decision Tree Visualization



d. Implement a Support Vector Machine for any relevant dataset.

Code :-

```

# =====
# 1. IMPORT LIBRARIES
# =====

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix

import matplotlib.pyplot as plt

import seaborn as sns

# =====

# 2. CREATE SAMPLE DATA

# =====

# Small dataset (like Iris)
  
```

```
data = {  
    'sepal_length': [5.1, 4.9, 5.8, 6.0, 6.3, 5.0, 5.5, 6.7, 6.5, 5.7],  
    'sepal_width': [3.5, 3.0, 2.7, 3.4, 3.3, 3.6, 2.8, 3.1, 2.8, 2.5],  
    'petal_length': [1.4, 1.4, 4.1, 4.5, 5.0, 1.5, 4.5, 5.6, 5.2, 4.0],  
    'petal_width': [0.2, 0.2, 1.3, 1.5, 2.0, 0.2, 1.5, 2.4, 2.0, 1.2],  
    'species':      ['setosa','setosa','versicolor','versicolor','virginica',  
                    'setosa','versicolor','virginica','virginica','versicolor']  
}
```

```
df = pd.DataFrame(data)
```

```
# =====
```

```
# 3. SPLIT FEATURES & TARGET
```

```
# =====
```

```
X = df.drop('species', axis=1)
```

```
y = df['species']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# =====
```

```
# 4. BUILD SVM MODEL
```

```
# =====
```

```
# Kernel can be 'linear', 'poly', 'rbf', 'sigmoid'
```

```
svm_model = SVC(kernel='linear', C=1.0, random_state=42)
```

```
svm_model.fit(X_train, y_train)
```

```
# =====
```

```
# 5. PREDICT & EVALUATE
```

```
# =====
```

```
y_pred = svm_model.predict(X_test)
```

```
print("Actual Labels:")
```

```
print(y_test.values)
```

```
print("\nPredicted Labels:")
```

```
print(y_pred)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"\nModel Accuracy: {accuracy*100:.2f}%")
```

```
# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6,4))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=svm_model.classes_,  
yticklabels=svm_model.classes_)
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```

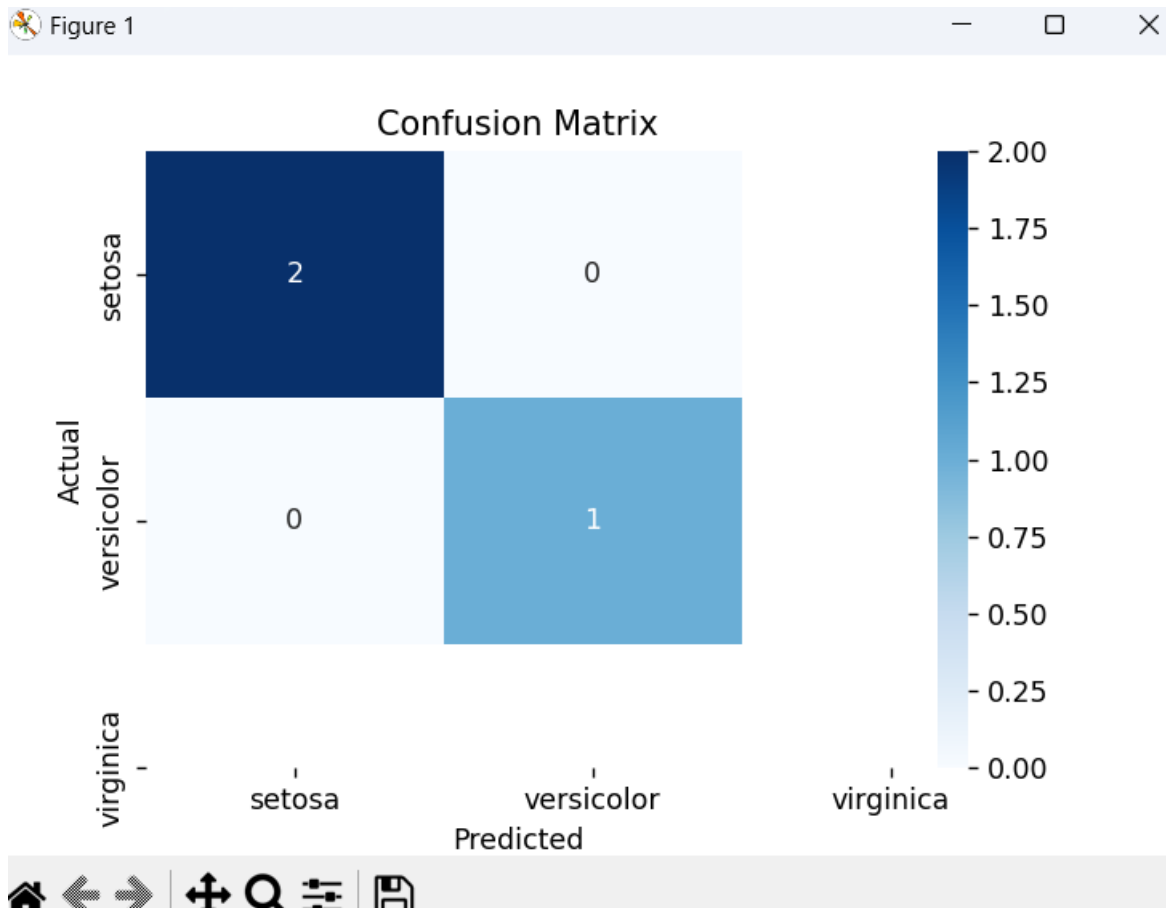
output :-

```
Actual Labels:  
['virginica' 'setosa' 'setosa']
```

```
Predicted Labels:  
['virginica' 'setosa' 'setosa']
```

```
Model Accuracy: 100.00%
```

```
|
```



e. Train a random forest ensemble. Experiment with the number of trees and feature sampling. Compare performance to a single decision tree.

Code :-

```
# =====
# 1. IMPORT LIBRARIES
# =====

import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

import matplotlib.pyplot as plt

# =====

# 2. CREATE SAMPLE DATA
```

```

# =====

data = {
    'sepal_length': [5.1, 4.9, 5.8, 6.0, 6.3, 5.0, 5.5, 6.7, 6.5, 5.7],
    'sepal_width': [3.5, 3.0, 2.7, 3.4, 3.3, 3.6, 2.8, 3.1, 2.8, 2.5],
    'petal_length': [1.4, 1.4, 4.1, 4.5, 5.0, 1.5, 4.5, 5.6, 5.2, 4.0],
    'petal_width': [0.2, 0.2, 1.3, 1.5, 2.0, 0.2, 1.5, 2.4, 2.0, 1.2],
    'species': ['setosa', 'setosa', 'versicolor', 'versicolor', 'virginica',
                'setosa', 'versicolor', 'virginica', 'virginica', 'versicolor']
}

df = pd.DataFrame(data)

# =====

# 3. SPLIT FEATURES & TARGET

# =====

X = df.drop('species', axis=1)
y = df['species']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# =====

# 4. SINGLE DECISION TREE

# =====

dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print(f"Decision Tree Accuracy: {accuracy_dt*100:.2f}%")

# =====

```

## # 5. RANDOM FOREST ENSEMBLE

# =====

# Experiment with number of trees and feature sampling

n\_trees\_list = [5, 10, 50]

max\_features\_list = [1, 2, 3, None]

for n\_trees in n\_trees\_list:

for max\_feat in max\_features\_list:

rf = RandomForestClassifier(

n\_estimators=n\_trees,

max\_features=max\_feat,

random\_state=42

)

rf.fit(X\_train, y\_train)

y\_pred\_rf = rf.predict(X\_test)

accuracy\_rf = accuracy\_score(y\_test, y\_pred\_rf)

print(f"Random Forest (trees={n\_trees}, max\_features={max\_feat}) Accuracy:  
{accuracy\_rf\*100:.2f}%")

# =====

## # 6. COMPARISON

# =====

plt.figure(figsize=(6,4))

plt.bar(['Decision Tree', 'RF 5', 'RF 10', 'RF 50'],

[accuracy\_dt,

accuracy\_score(y\_test, RandomForestClassifier(n\_estimators=5, random\_state=42).fit(X\_train,  
y\_train).predict(X\_test)),

accuracy\_score(y\_test, RandomForestClassifier(n\_estimators=10, random\_state=42).fit(X\_train,  
y\_train).predict(X\_test)),

accuracy\_score(y\_test, RandomForestClassifier(n\_estimators=50, random\_state=42).fit(X\_train,  
y\_train).predict(X\_test))])

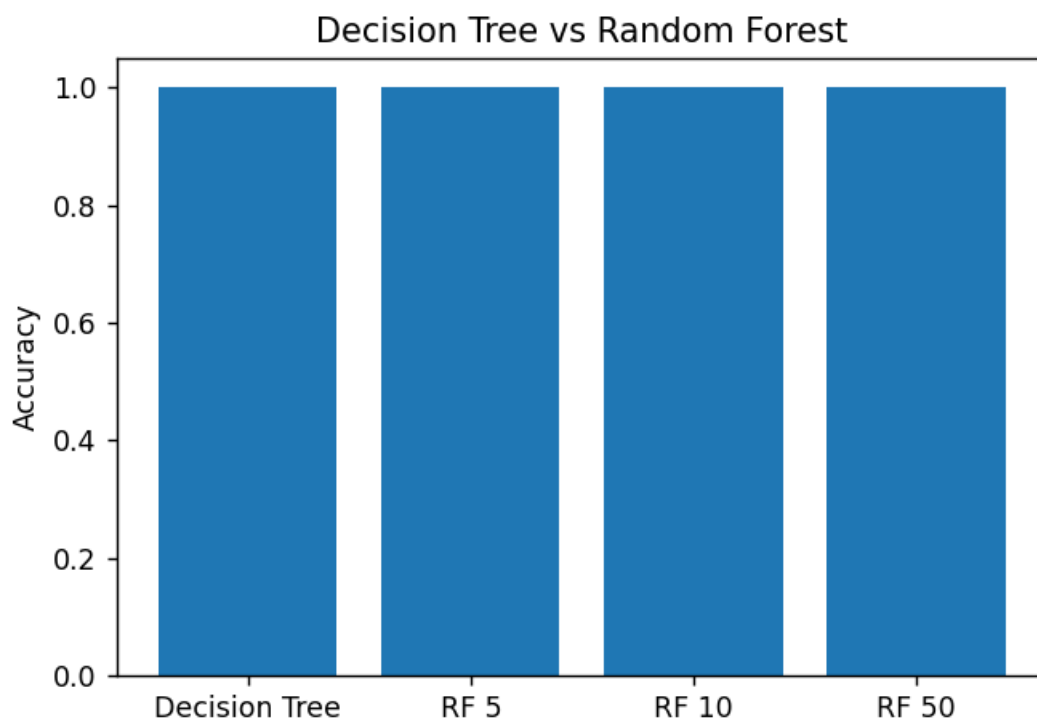
```
plt.ylabel("Accuracy")
```

```
plt.title("Decision Tree vs Random Forest")
```

```
plt.show()
```

output :-

```
Decision Tree Accuracy: 100.00%
Random Forest (trees=5, max_features=1) Accuracy: 66.67%
Random Forest (trees=5, max_features=2) Accuracy: 100.00%
Random Forest (trees=5, max_features=3) Accuracy: 66.67%
Random Forest (trees=5, max_features=None) Accuracy: 66.67%
Random Forest (trees=10, max_features=1) Accuracy: 66.67%
Random Forest (trees=10, max_features=2) Accuracy: 100.00%
Random Forest (trees=10, max_features=3) Accuracy: 66.67%
Random Forest (trees=10, max_features=None) Accuracy: 66.67%
Random Forest (trees=50, max_features=1) Accuracy: 100.00%
Random Forest (trees=50, max_features=2) Accuracy: 100.00%
Random Forest (trees=50, max_features=3) Accuracy: 100.00%
Random Forest (trees=50, max_features=None) Accuracy: 66.67%
```



f. Implement a gradient boosting machine (e.g., XGBoost). Tune hyperparameters and explore feature importance.

Code :-

```
# =====  
  
# XGBoost Classifier on Iris Dataset  
  
# =====  
  
import pandas as pd  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.preprocessing import LabelEncoder  
  
import xgboost as xgb  
  
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report  
  
  
# Load dataset  
  
from sklearn.datasets import load_iris  
  
iris = load_iris()  
  
X = iris.data  
  
y = iris.target # Already numeric in sklearn dataset, but let's assume you had strings  
  
  
# If y is string labels like 'setosa', 'versicolor', 'virginica'  
  
# Example: uncomment below if using a CSV  
  
# y = df['species'] # string labels  
  
# le = LabelEncoder()  
  
# y = le.fit_transform(y)  
  
  
# Split into train and test  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
  
  
# Initialize XGBoost classifier  
  
xgb_model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='mlogloss') # avoid warning  
  
  
# Train model  
  
xgb_model.fit(X_train, y_train)
```

```
# Predict
```

```
y_pred = xgb_model.predict(X_test)
```

```
# Evaluate
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
# Optional: if you used LabelEncoder and want original string labels
```

```
# y_pred_labels = le.inverse_transform(y_pred)
```

```
# print(y_pred_labels)
```

Output :-

```
Accuracy: 1.0
```

```
Confusion Matrix:
```

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
Classification Report:
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 10      |
| 1            | 1.00      | 1.00   | 1.00     | 9       |
| 2            | 1.00      | 1.00   | 1.00     | 11      |
| accuracy     |           |        | 1.00     | 30      |
| macro avg    | 1.00      | 1.00   | 1.00     | 30      |
| weighted avg | 1.00      | 1.00   | 1.00     | 30      |

## Practical - 5

### 5. Generative Models

a. Implement and demonstrate the working of a Naive Bayesian classifier using a sample data set. Build the model to classify a test sample.

Code :-

```
import math
```

```
from collections import defaultdict
```

```
class NaiveBayesClassifier:
```

```
    def __init__(self):
```

```
        self.class_probs = {}
```

```
        self.feature_probs = {}
```

```
        self.classes = set()
```

```
    def fit(self, X, y):
```

```
        """
```

```
        X: list of feature dictionaries
```

```
        y: list of labels
```

```
        """
```

```
        n = len(y)
```

```
        self.classes = set(y)
```

```
        # Count occurrences
```

```
        class_counts = defaultdict(int)
```

```
        feature_counts = {c: defaultdict(lambda: defaultdict(int)) for c in self.classes}
```

```
        for xi, label in zip(X, y):
```

```
            class_counts[label] += 1
```

```
            for feature, value in xi.items():
```

```
feature_counts[label][feature][value] += 1
```

```
# Compute class probabilities P(class)
```

```
self.class_probs = {c: class_counts[c]/n for c in self.classes}
```

```
# Compute conditional probabilities P(feature=value | class)
```

```
self.feature_probs = {c: {} for c in self.classes}
```

```
for c in self.classes:
```

```
    for feature in feature_counts[c]:
```

```
        total = sum(feature_counts[c][feature].values())
```

```
        self.feature_probs[c][feature] = {
```

```
            val: (count/total) for val, count in feature_counts[c][feature].items()
```

```
        }
```

```
def predict(self, x):
```

```
    """
```

```
    x: dictionary of feature:value
```

```
    returns: predicted class
```

```
    """
```

```
    class_scores = {}
```

```
    for c in self.classes:
```

```
        # Start with prior P(class)
```

```
        score = math.log(self.class_probs[c])
```

```
        for feature, value in x.items():
```

```
            if value in self.feature_probs[c].get(feature, {}):
```

```
                score += math.log(self.feature_probs[c][feature][value])
```

```
            else:
```

```
                # Handle unseen feature values with smoothing
```

```
                score += math.log(1e-6)
```

```
    class_scores[c] = score
```

```

# Return class with max score

return max(class_scores, key=class_scores.get)

# -----

# Example dataset: Weather & Play

# -----

X = [
    {"outlook": "sunny", "temp": "hot", "humidity": "high", "wind": "weak"},
    {"outlook": "sunny", "temp": "hot", "humidity": "high", "wind": "strong"},
    {"outlook": "overcast", "temp": "hot", "humidity": "high", "wind": "weak"},
    {"outlook": "rain", "temp": "mild", "humidity": "high", "wind": "weak"},
    {"outlook": "rain", "temp": "cool", "humidity": "normal", "wind": "weak"},
    {"outlook": "rain", "temp": "cool", "humidity": "normal", "wind": "strong"},
    {"outlook": "overcast", "temp": "cool", "humidity": "normal", "wind": "strong"},
]

y = ["no", "no", "yes", "yes", "yes", "no", "yes"]

# Train model

nb = NaiveBayesClassifier()

nb.fit(X, y)

# Test sample

test_sample = {"outlook": "sunny", "temp": "cool", "humidity": "high", "wind": "strong"}

prediction = nb.predict(test_sample)

print("Test Sample:", test_sample)

print("Predicted Class:", prediction)

output :-

```

```
Test Sample: {'outlook': 'sunny', 'temp': 'cool', 'humidity': 'high', 'wind': 'strong'}  
Predicted Class: no
```

---

## b. Implement Hidden Markov Models using hmmlearn

code :-

```
# =====  
  
# 1. IMPORT LIBRARIES  
  
# =====  
  
import numpy as np  
from hmmlearn import hmm  
  
# =====  
  
# 2. CREATE SYNTHETIC OBSERVATION DATA  
  
# =====  
  
# Suppose we have 2 hidden states and 3 possible observable symbols (0, 1, 2)  
# Example: Weather (hidden state) -> Observations (activity)  
# States: 0 = Rainy, 1 = Sunny  
# Observations: 0 = Walk, 1 = Shop, 2 = Clean  
  
# Define observation sequences (training data)  
observations = np.array([[0, 1, 2, 1, 0, 1, 2, 1, 0, 2]]).T # Shape (n_samples, 1)  
  
# =====  
  
# 3. INITIALIZE HMM  
  
# =====  
  
model = hmm.MultinomialHMM(n_components=2, n_iter=100, random_state=42)  
  
# =====  
  
# 4. FIT THE MODEL  
  
# =====  
  
model.fit(observations)
```

```

# =====

# 5. PRINT MODEL PARAMETERS

# =====

print("Transition matrix (hidden states probabilities):")

print(model.transmat_)

print("\nEmission matrix (observation probabilities):")

print(model.emissionprob_)

print("\nStart probabilities (initial state probabilities):")

print(model.startprob_)

# =====

# 6. PREDICT HIDDEN STATES

# =====

hidden_states = model.predict(observations)

print("\nPredicted Hidden States:")

print(hidden_states)

# =====

# 7. GENERATE A SEQUENCE FROM HMM

# =====

X, Z = model.sample(10) # Generate 10 observations and hidden states

print("\nGenerated Observation Sequence:")

print(X.ravel())

print("Corresponding Hidden States:")

print(Z)

output :-

```

Transition matrix (hidden states probabilities):

```
[[0.87827377 0.12172623]
 [0.99887354 0.00112646]]
```

Emission matrix (observation probabilities):

```
[[1.]
 [1.]]
```

Start probabilities (initial state probabilities):

```
[0.17550431 0.82449569]
```

Predicted Hidden States:

```
[1 0 0 0 0 0 0 0 0]
```

## Practical - 6

### 6. Probabilistic Models

a. Implement Bayesian Linear Regression to explore prior and posterior distribution.

Code :-

```
import numpy as np
import matplotlib.pyplot as plt

# Seed for reproducibility
np.random.seed(42)

# Create synthetic data
def generate_data(n_samples=10):
    X = np.linspace(0, 1, n_samples)
    y = 2 * X + 1 + 0.1 * np.random.randn(n_samples)
    return X.reshape(-1, 1), y

# Add bias term
def design_matrix(X):
    return np.hstack([np.ones((X.shape[0], 1)), X])

# Bayesian Linear Regression Posterior
def compute_posterior(X, y, sigma2, prior_mean, prior_cov):
    XT_X = X.T @ X
    sigma_inv = np.linalg.inv(prior_cov)
    Sigma_n = np.linalg.inv(sigma_inv + XT_X / sigma2)
    mu_n = Sigma_n @ (sigma_inv @ prior_mean + X.T @ y / sigma2)
    return mu_n, Sigma_n

# Sample from posterior
```

```
def sample_posterior(mu, cov, n_samples=5):  
    return np.random.multivariate_normal(mu, cov, n_samples)  
  
# Generate data  
X_raw, y = generate_data(n_samples=10)  
X = design_matrix(X_raw)  
  
# Prior: zero-mean Gaussian  
prior_mean = np.zeros(X.shape[1])  
prior_cov = np.eye(X.shape[1]) * 1.0  
  
# Noise variance  
sigma2 = 0.01  
  
# Posterior  
mu_n, Sigma_n = compute_posterior(X, y, sigma2, prior_mean, prior_cov)  
  
# Plotting  
x_plot = np.linspace(0, 1, 100).reshape(-1, 1)  
X_plot = design_matrix(x_plot)  
  
# Plot samples from prior  
prior_samples = sample_posterior(prior_mean, prior_cov, n_samples=5)  
plt.figure(figsize=(12, 5))  
  
plt.subplot(1, 2, 1)  
for w in prior_samples:  
    y_plot = X_plot @ w  
    plt.plot(x_plot, y_plot, lw=1)  
plt.title("Samples from Prior")
```

```
plt.xlabel("x")
```

```
plt.ylabel("y")
```

```
# Plot samples from posterior
```

```
posterior_samples = sample_posterior(mu_n, Sigma_n, n_samples=5)
```

```
plt.subplot(1, 2, 2)
```

```
for w in posterior_samples:
```

```
    y_plot = X_plot @ w
```

```
    plt.plot(x_plot, y_plot, lw=1)
```

```
plt.scatter(X_raw, y, c='red', label='Data')
```

```
plt.title("Samples from Posterior")
```

```
plt.xlabel("x")
```

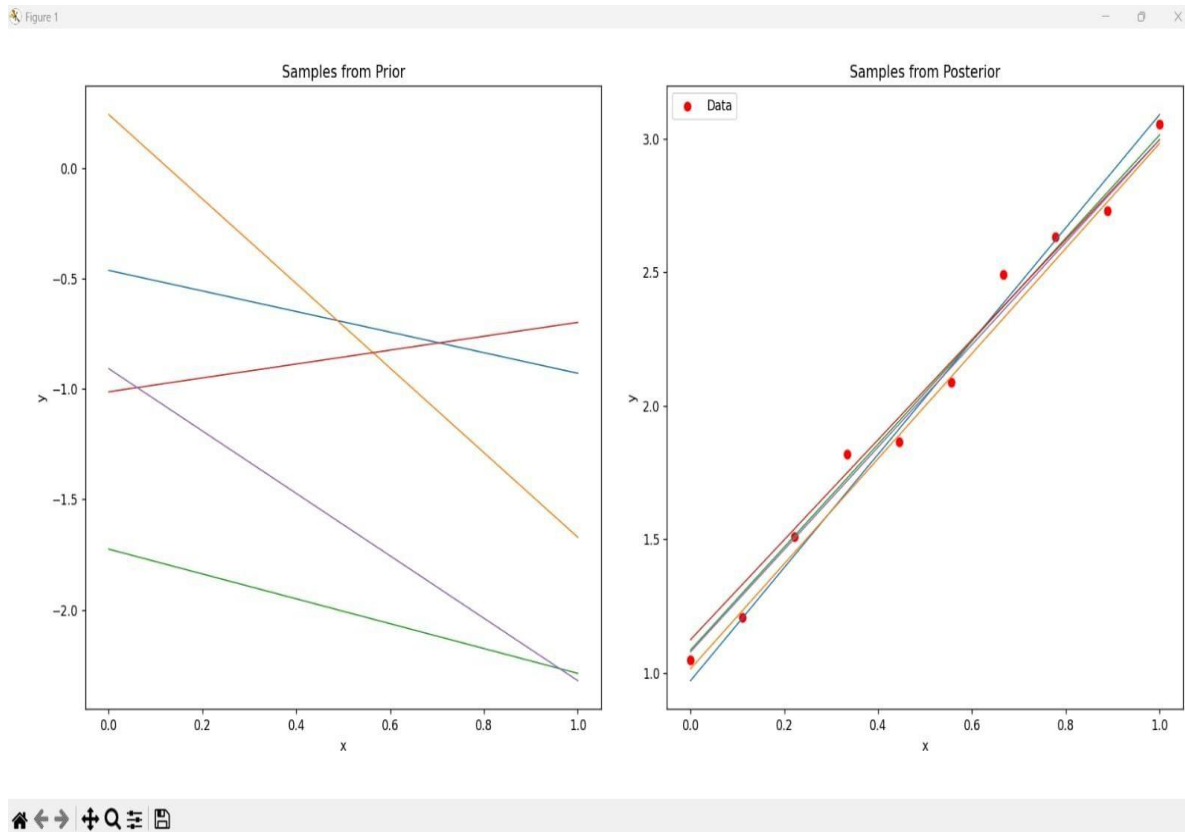
```
plt.ylabel("y")
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

output :-



b. Implement Gaussian Mixture Models for density estimation and unsupervised clustering.

Code :-

# Import required libraries

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.mixture import GaussianMixture
```

```
from sklearn.datasets import make_blobs
```

# Step 1: Generate synthetic data

```
X, y_true = make_blobs(n_samples=500, centers=3, cluster_std=1.0, random_state=42)
```

# Step 2: Fit a Gaussian Mixture Model

```
gmm = GaussianMixture(n_components=3, covariance_type='full', random_state=42)
```

```
gmm.fit(X)
```

```
labels = gmm.predict(X) # Cluster assignments
```

# Step 3: Plot the clusters

```
plt.figure(figsize=(8,6))
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=40)
```

```
plt.scatter(gmm.means_[:, 0], gmm.means_[:, 1], c='red', marker='x', s=100, label='Centroids')

plt.title("Gaussian Mixture Model Clustering")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.legend()

plt.show()

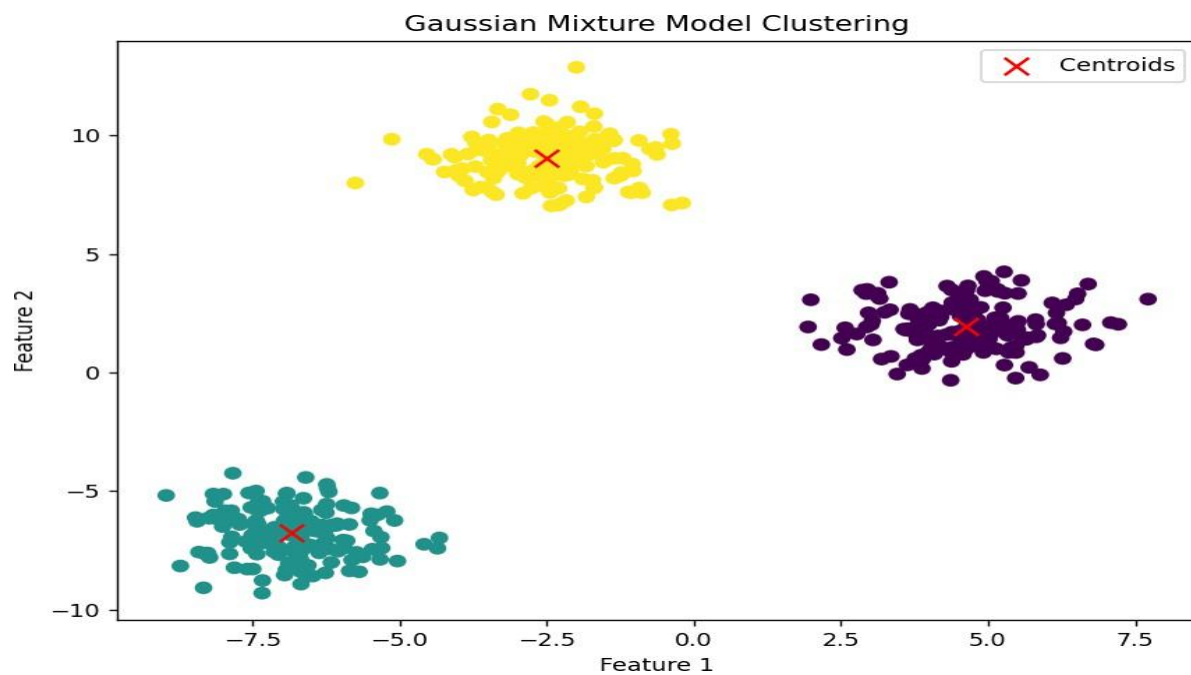
# Step 4: Display probabilities for each sample (density estimation)

probabilities = gmm.predict_proba(X)

print("First 5 samples probabilities:\n", probabilities[:5])
```

output :-

```
First 5 samples probabilities:
[[5.75348038e-41 1.00000000e+00 5.20611097e-66]
 [1.00000000e+00 9.49955967e-41 2.07026456e-15]
 [1.00000000e+00 5.59282634e-71 3.15035295e-34]
 [7.78906727e-19 1.29108364e-50 1.00000000e+00]
 [1.00000000e+00 1.26714620e-65 1.84345162e-29]]
```



## Practical - 7

### 7. Model Evaluation and Hyperparameter Tuning

#### a. Implement cross-validation techniques (k-fold, stratified, etc.) for robust model evaluation

code :-

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import KFold, StratifiedKFold, cross_val_score

from sklearn.linear_model import LogisticRegression

#Load dataset

data = load_iris()

X, y = data.data, data.target

#Model

model = LogisticRegression(max_iter=200)

#1. K-Fold Cross Validation

print("K-Fold Cross Validation:")

kf = KFold(n_splits=5, shuffle=True, random_state=42)

scores_kf = cross_val_score(model, X, y, cv=kf)

print("Scores:", scores_kf)

print("Mean Accuracy:", np.mean(scores_kf))

print()

#2. Stratified K-Fold Cross Validation

print("Stratified K-Fold Cross Validation:")

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores_skf = cross_val_score(model, X, y, cv=skf)

print("Scores:", scores_skf)

print("Mean Accuracy:", np.mean(scores_skf))

#3. Default cross_val_score with StratifiedKFold for classification

print("\nDefault 5-Fold (Stratified for classification):")
```

```
scores_default = cross_val_score(model, X, y, cv=5)

print("Scores:", scores_default)

print("Mean Accuracy:", np.mean(scores_default))
```

output :-

```
K-Fold Cross Validation:
Scores: [1.          1.          0.93333333 0.96666667 0.96666667]
Mean Accuracy: 0.9733333333333334

Stratified K-Fold Cross Validation:
Scores: [1.          0.96666667 0.93333333 1.          0.93333333]
Mean Accuracy: 0.9666666666666668

Default 5-Fold (Stratified for classification):
Scores: [0.96666667 1.          0.93333333 0.96666667 1.          ]
Mean Accuracy: 0.9733333333333334
```

b. Systematically explore combinations of hyperparameters to optimize model performance.(use grid and randomized search)

code :-

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

from sklearn.linear_model import LogisticRegression

from scipy.stats import uniform

#Load dataset

data = load_iris()

X, y = data.data, data.target

#Grid Search CV (fixed)

print("Grid Search CV:")

param_grid = [

    {'solver': ['lbfgs'], 'penalty': ['l2'], 'C': [0.01, 0.1, 1, 10, 100]},

    {'solver': ['saga'], 'penalty': ['l1', 'l2'], 'C': [0.01, 0.1, 1, 10, 100]}

]

grid = GridSearchCV(LogisticRegression(max_iter=500), param_grid, cv=5, scoring='accuracy', n_jobs=1)

grid.fit(X, y)
```

```
print("Best Parameters (GridSearch):", grid.best_params_)

print("Best CV Score:", grid.best_score_)

print()

#Randomized Search CV (fixed)

print("Randomized Search CV:")

param_dist = {

    'C': uniform(loc=0.01, scale=10),

    'solver': ['saga', 'lbfgs'],

    'penalty': ['l1', 'l2'] # saga supports l1, l2; lbfgs only l2 (handled automatically)

}

random_search = RandomizedSearchCV(LogisticRegression(max_iter=500),

param_distributions=param_dist, n_iter=20, cv=5, scoring='accuracy', random_state=42, n_jobs=1)

random_search.fit(X, y)

print("Best Parameters (RandomizedSearch):", random_search.best_params_)

print("Best CV Score:", random_search.best_score_)

output :-
```

```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:\Users\ASUS\OneDrive\Documents\Priyanka\Systematically hyperparameter.py
Grid Search CV:

Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model\_sag.py", line 348
    warnings.warn(
ConvergenceWarning: The max_iter was reached which means the coef_ did not converge

Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model\_sag.py", line 348
    warnings.warn(
ConvergenceWarning: The max_iter was reached which means the coef_ did not converge

Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model\_sag.py", line 348
    warnings.warn(
ConvergenceWarning: The max_iter was reached which means the coef_ did not converge

Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model\_sag.py", line 348
    warnings.warn(
ConvergenceWarning: The max_iter was reached which means the coef_ did not converge

Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model\_sag.py", line 348
    warnings.warn(
ConvergenceWarning: The max_iter was reached which means the coef_ did not converge

Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model\_sag.py", line 348
    warnings.warn(
ConvergenceWarning: The max_iter was reached which means the coef_ did not converge

Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model\_sag.py", line 348
    warnings.warn(
ConvergenceWarning: The max_iter was reached which means the coef_ did not converge

```



## Practical - 8

### 8. Bayesian Learning

#### a. Implement Bayesian Learning using inferences

code :-

```
# Bayesian Learning in Python: Two Examples
```

```
# 1) Parameter inference with a Beta-Binomial model
```

```
# 2) Classification with Multinomial Naive Bayes (Dirichlet prior / Laplace smoothing)
```

```
# WITHOUT using pandas
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy.stats import beta
```

```
from collections import Counter
```

```
from math import log, exp
```

```
# --- Part 1: Beta-Binomial (learning the bias of a coin) ---
```

```
alpha_prior, beta_prior = 2, 2 # Prior: Beta(alpha, beta)
```

```
# Observed data: 1=heads, 0=tails
```

```
observations = np.array([1,1,0,1,0,1,1,1,0,1, 1,0,1,1,1,0,1,1,1,1])
```

```
n_heads = observations.sum()
```

```
n_tails = len(observations) - n_heads
```

```
# Posterior parameters
```

```
alpha_post = alpha_prior + n_heads
```

```
beta_post = beta_prior + n_tails
```

```
# Posterior summaries
```

```
posterior_mean = alpha_post / (alpha_post + beta_post)
```

```
posterior_map = (alpha_post - 1) / (alpha_post + beta_post - 2) if (alpha_post>1 and beta_post>1) else  
np.nan
```

```
predictive_next_head = posterior_mean
```

```
print("=== Beta-Binomial Inference (Coin Bias) ===")
```

```
print(f"Prior Beta({alpha_prior}, {beta_prior})")
```

```
print(f"Observed: {len(observations)} flips => heads={n_heads}, tails={n_tails}")
```

```
print(f"Posterior Beta({alpha_post}, {beta_post})")
```

```
print(f"Posterior mean of p(heads): {posterior_mean:.4f}")
```

```
print(f"Posterior MAP of p(heads): {posterior_map:.4f}")
```

```
print(f"Posterior predictive P(next flip = heads): {predictive_next_head:.4f}\n")
```

```
# Plot prior and posterior densities
```

```
x = np.linspace(0, 1, 500)
```

```
plt.figure()
```

```
plt.plot(x, beta.pdf(x, alpha_prior, beta_prior), label=f"Prior Beta({alpha_prior},{beta_prior})")
```

```
plt.plot(x, beta.pdf(x, alpha_post, beta_post), label=f"Posterior Beta({alpha_post},{beta_post})")
```

```
plt.title("Prior vs Posterior over coin bias p")
```

```
plt.xlabel("p (probability of heads)")
```

```
plt.ylabel("Density")
```

```
plt.legend()
```

```
plt.show()
```

```
# --- Part 2: Multinomial Naive Bayes (Dirichlet prior) ---
```

```
train_docs = [
```

```
    ("spam", "win cash now"),
```

```
    ("spam", "limited offer claim prize"),
```

```
    ("spam", "win big prize cash"),
```

```
    ("ham", "meeting at noon"),
```

```
("ham", "let us schedule a meeting"),  
("ham", "please review the report"),  
("ham", "see you at noon"),  
]
```

```
test_docs = [  
    ("?", "win a prize now"),  
    ("?", "see you at the meeting"),  
    ("?", "claim your cash offer"),  
    ("?", "review the schedule"),  
]
```

```
def tokenize(s):  
    return s.lower().split()
```

```
# Build vocabulary
```

```
classes = sorted(set(lbl for lbl, _ in train_docs))  
vocab = sorted(set(w for _, text in train_docs for w in tokenize(text)))  
word_to_idx = {w:i for i,w in enumerate(vocab)}
```

```
alpha = 1.0 # Dirichlet prior smoothing
```

```
# Count words per class
```

```
class_doc_counts = Counter(lbl for lbl, _ in train_docs)  
class_token_counts = {c: np.zeros(len(vocab), dtype=np.int64) for c in classes}  
class_total_tokens = {c: 0 for c in classes}
```

```
for lbl, text in train_docs:
```

```
    toks = tokenize(text)
```

```
    for t in toks:
```

```
class_token_counts[lbl][word_to_idx[t]] += 1
```

```
class_total_tokens[lbl] += 1
```

```
# Class prior with smoothing
```

```
alpha_class = 1.0
```

```
num_classes = len(classes)
```

```
total_docs = len(train_docs)
```

```
class_prior_logprob = {}
```

```
for c in classes:
```

```
    class_prior_logprob[c] = log((class_doc_counts[c] + alpha_class) /  
                                (total_docs + num_classes * alpha_class))
```

```
# Conditional probabilities  $P(w|c)$ 
```

```
cond_prob = {}
```

```
for c in classes:
```

```
    counts = class_token_counts[c].astype(float)
```

```
    denom = class_total_tokens[c] + alpha * len(vocab)
```

```
    cond_prob[c] = (counts + alpha) / denom
```

```
def doc_log_posteriors(text):
```

```
    toks = tokenize(text)
```

```
    token_counts = Counter(toks)
```

```
    logps = {}
```

```
    for c in classes:
```

```
        logp = class_prior_logprob[c]
```

```
        for w, cnt in token_counts.items():
```

```
            if w in word_to_idx:
```

```
                logp += cnt * log(cond_prob[c][word_to_idx[w]])
```

```
            else:
```

```
                denom = class_total_tokens[c] + alpha * len(vocab)
```

```

        logp += cnt * log(alpha / denom)

    logps[c] = logp

maxlog = max(logps.values())

exps = {c: exp(logps[c] - maxlog) for c in classes}

Z = sum(exps.values())

return {c: exps[c] / Z for c in classes}

print("=== Multinomial Naive Bayes (Dirichlet prior) ===")

print("Vocabulary:", vocab)

print("\nPosterior predictions on test documents:")


# Print table header

print(f"{'Text':35s} {'P(spam)':>10s} {'P(ham)':>10s} {'Prediction':>12s}")

print("-"*70)


# Print predictions row by row

for _, text in test_docs:

    post = doc_log_posteriors(text)

    pred = max(post.items(), key=lambda kv: kv[1])[0]

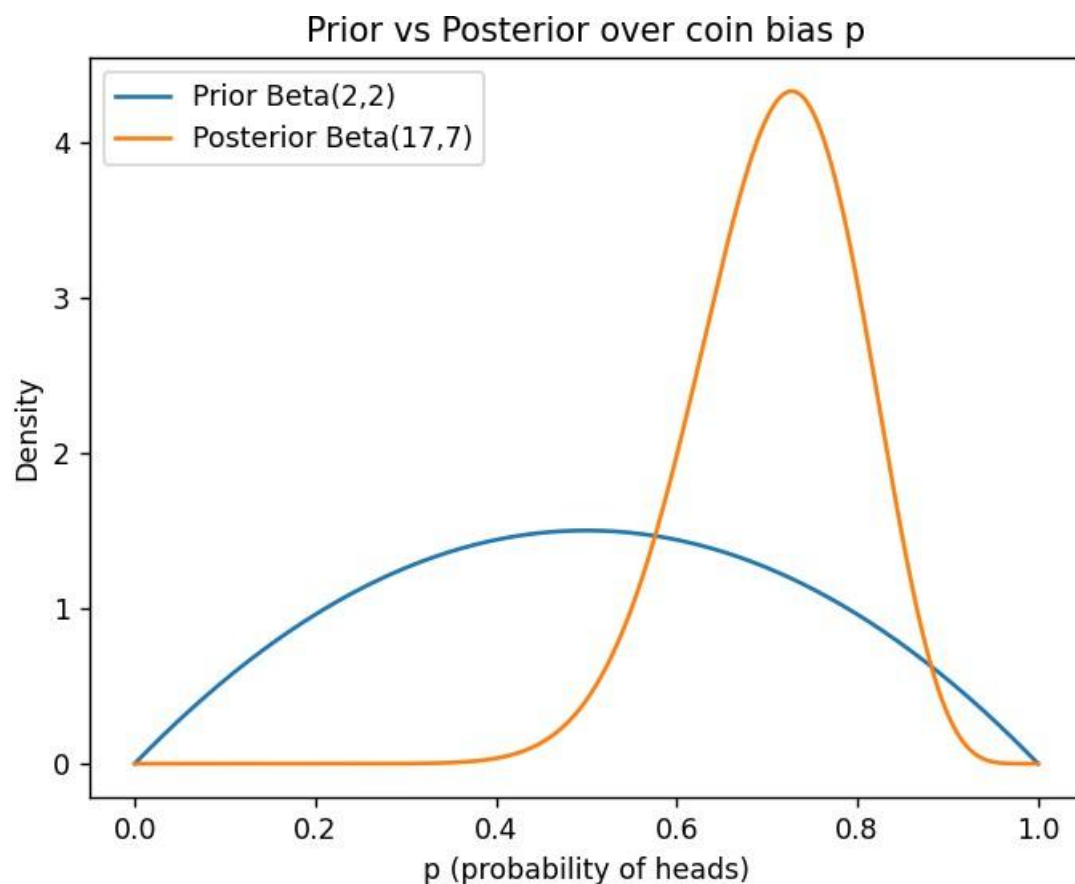
    print(f"{text:35s} {post['spam']:.4f} {post['ham']:.4f} {pred:>12s}")

output :-

```

```
=== Beta-Binomial Inference (Coin Bias) ===  
Prior Beta(2, 2)  
Observed: 20 flips => heads=15, tails=5  
Posterior Beta(17, 7)  
Posterior mean of p(heads): 0.7083  
Posterior MAP of p(heads): 0.7273  
Posterior predictive P(next flip = heads): 0.7083
```

1



## Practical - 9

### 9. Deep Generative Models

a. Set up a generator network to produce samples and a discriminator network to distinguish between real and generated data. (Use a simple small dataset)

code:-

```
import random

import matplotlib.pyplot as plt

# Real data distribution: mean = 4

def sample_real():

    return random.gauss(4, 1)

# Generator: starts with noise and learns a shift

class Generator:

    def __init__(self):

        self.bias = random.uniform(-1, 1) # this is the "parameter" it learns

    def generate(self):

        noise = random.uniform(-1, 1)

        return noise + self.bias

    def update(self, gradient, lr=0.01):

        self.bias += lr * gradient

# Discriminator: simple linear classifier

class Discriminator:

    def __init__(self):

        self.threshold = 0.0 # threshold to separate real and fake

    def predict(self, x):
```

```
return 1 if x > self.threshold else 0 # 1 = real, 0 = fake
```

```
def update(self, real_samples, fake_samples):
```

```
    real_mean = sum(real_samples) / len(real_samples)
```

```
    fake_mean = sum(fake_samples) / len(fake_samples)
```

```
    self.threshold = (real_mean + fake_mean) / 2 # separate boundary
```

```
# Instantiate
```

```
G = Generator()
```

```
D = Discriminator()
```

```
# Training loop
```

```
for epoch in range(500):
```

```
    real_data = [sample_real() for _ in range(10)]
```

```
    fake_data = [G.generate() for _ in range(10)]
```

```
# Update Discriminator
```

```
D.update(real_data, fake_data)
```

```
# Generator loss: wants D(fake) = 1
```

```
grad = 0.0
```

```
for x in fake_data:
```

```
    prediction = D.predict(x)
```

```
    grad += (1 - prediction) # if D says fake (0), G wants to improve
```

```
grad /= len(fake_data)
```

```
G.update(grad)
```

```
if epoch % 50 == 0:
```

```
    print(f"Epoch {epoch} | Generator bias: {G.bias:.4f} | Discriminator threshold: {D.threshold:.4f}")
```

```
# Plot results
```

```
generated_samples = [G.generate() for _ in range(1000)]
```

```
real_samples = [sample_real() for _ in range(1000)]
```

```
plt.hist(real_samples, bins=40, alpha=0.5, label="Real")
```

```
plt.hist(generated_samples, bins=40, alpha=0.5, label="Generated")
```

```
plt.legend()
```

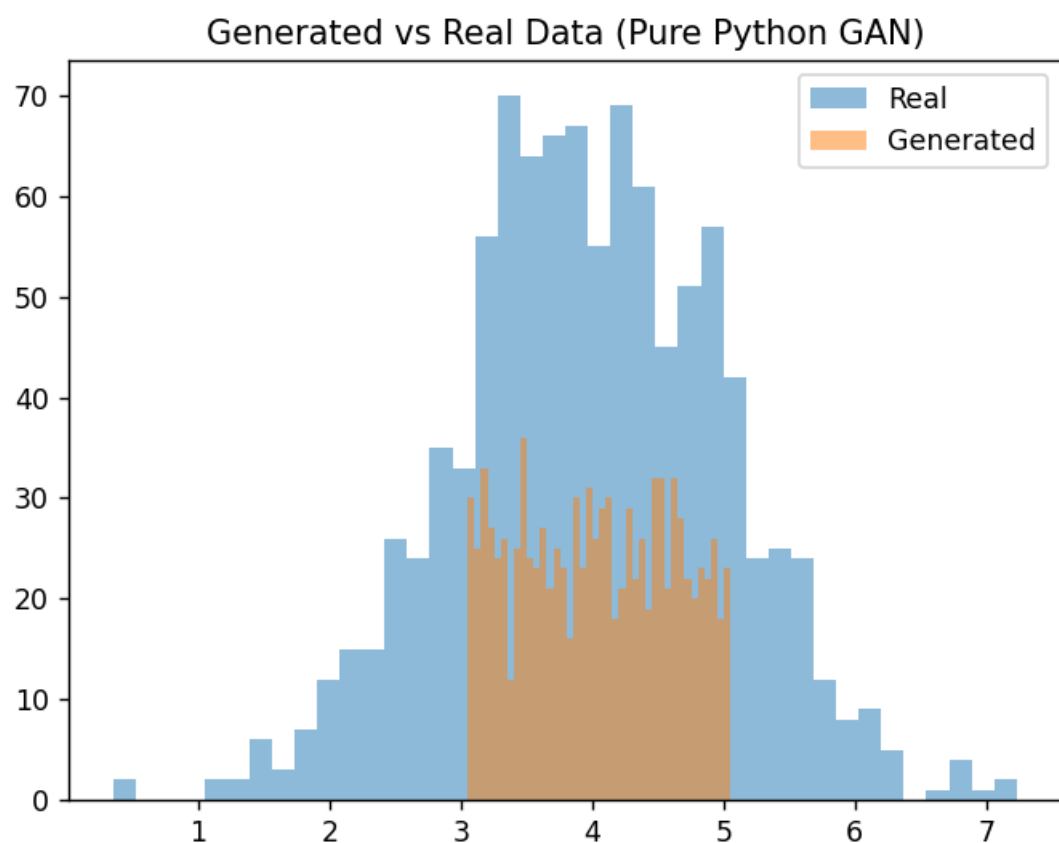
```
plt.title("Generated vs Real Data (Pure Python GAN)")
```

```
plt.show()
```

output :-

```
Epoch 0 | Generator bias: -0.1349 | Discriminator threshold: 1.6529
Epoch 50 | Generator bias: 0.3651 | Discriminator threshold: 2.3046
Epoch 100 | Generator bias: 0.8651 | Discriminator threshold: 2.4751
Epoch 150 | Generator bias: 1.3651 | Discriminator threshold: 2.7668
Epoch 200 | Generator bias: 1.8641 | Discriminator threshold: 3.2583
Epoch 250 | Generator bias: 2.3451 | Discriminator threshold: 3.0061
Epoch 300 | Generator bias: 2.7861 | Discriminator threshold: 3.4552
Epoch 350 | Generator bias: 3.1741 | Discriminator threshold: 3.6269
Epoch 400 | Generator bias: 3.5131 | Discriminator threshold: 3.5384
Epoch 450 | Generator bias: 3.7941 | Discriminator threshold: 3.7555
|
```

Figure 1



(x, y) = (6.81, 62.2)

## Practical – 10

10. Develop an API to deploy your model and perform predictions

Code :-

```
import random

import json

from http.server import BaseHTTPRequestHandler, HTTPServer

from urllib.parse import urlparse, parse_qs
```

# === Simple Generator class ===

```
class Generator:
```

```
    def __init__(self):
```

```
        self.bias = 3.5 # Pretend this is a trained parameter
```

```
    def generate(self, n=1):
```

```
        return [random.uniform(-1, 1) + self.bias for _ in range(n)]
```

# Instantiate the generator

```
gen = Generator()
```

# === HTTP Request Handler ===

```
class SimpleRequestHandler(BaseHTTPRequestHandler):
```

```
    def do_GET(self):
```

```
        parsed_path = urlparse(self.path)
```

```
        if parsed_path.path == '/generate':
```

```
            # Parse query parameters
```

```
            query = parse_qs(parsed_path.query)
```

```
            n = int(query.get('n', [1])[0]) # default n=1
```

```
            # Generate data
```

```
            samples = gen.generate(n)
```

```
# Send response  
self.send_response(200)  
self.send_header('Content-Type', 'application/json')  
self.end_headers()
```

```
response = {'generated_data': samples}  
self.wfile.write(json.dumps(response).encode())  
else:  
    self.send_error(404, 'Not Found')
```

```
# === Run Server ===
```

```
def run(server_class=HTTPServer, handler_class=SimpleRequestHandler, port=8000):  
    server_address = ("", port)  
    httpd = server_class(server_address, handler_class)  
    print(f'Serving on http://localhost:{port}')  
    httpd.serve_forever()
```

```
if __name__ == '__main__':  
    run()
```

output :-

Serving on http://localhost:8000